

BÁO CÁO THỰC HÀNH AN TOÀN ỨNG DỤNG WEB

LAB 2

BUFFER OVERFLOW TRONG ỨNG DỤNG WEB LOCAL

Từ HTTP POST tới stack frame, AddressSanitizer và bản vá

MSSV: 21127645

Họ tên: Lê Minh

Ngày lập báo cáo: 14/07/2026

Mục lục

Mục lục	2
Chương 1. Giới thiệu	5
Bối cảnh và mục tiêu	5
Phạm vi và an toàn	5
01_home_overview.png	5
Chương 2. Cơ sở lý thuyết	5
Bộ nhớ và chuỗi C	5
API xử lý chuỗi	6
Công cụ và hardening	6
Chương 3. Kiến trúc hệ thống	6
Ba tầng	6
Sequence diagram	6
Chương 4. Thiết kế chương trình lỗi	7
Mã nguồn và nguyên nhân	7
Chương 5. Thực nghiệm input bình thường	7
Kịch bản Le Minh	7
02_normal_input_before_submit.png	7
03_normal_http_request.png	8
04_normal_native_process.png	8
05_normal_memory_visualizer.png	8
Chương 6. Thực nghiệm input dài	8
Các mốc	8
Giới hạn kết luận	8
06_overflow_32_input.png	9
07_overflow_32_memory_boundary.png	9
08_overflow_64_request.png	9
09_overflow_64_strcpy_step.png	10
10_overflow_64_memory_visualizer.png	10
11_asan_detected.png	10
12_asan_stack_trace.png	10
13_native_crash_result.png	11

14_final_vulnerable_verdict.png	11
15_length_test_table.png	11
Chương 7. Phân tích GDB	11
Quy trình quan sát	11
Giới hạn bằng chứng	11
16_gdb_breakpoint.png	12
17_gdb_local_buffer.png	12
18_gdb_overflow_stop.png	12
Chương 8. Bản vá 1 - kiểm tra độ dài	12
Thiết kế	12
19_secure_length_reject.png	13
20_secure_length_timeline.png	13
Chương 9. Bản vá 2 - snprintf	13
Thiết kế	13
21_secure_snprintf_reject.png	13
22_code_comparison.png	14
Chương 10. Request limit	14
Defense in depth	14
Chương 11. Compiler hardening	14
Cơ chế	14
23_hardening_comparison.png	15
24_stack_canary_explanation.png	15
25_asan_vs_hardening.png	15
Chương 12. So sánh trước và sau vá	15
Kết quả thiết kế	15
Chương 13. Mức độ ảnh hưởng	16
Tác động	16
Chương 14. Phòng chống	16
Biện pháp	16
Chương 15. Trả lời câu hỏi báo cáo	16
1. Buffer Overflow khác Injection như thế nào?	16
2. Vì sao backend native bị kích hoạt qua HTTP?	16

3. Vì sao firewall không đủ?	16
4. Bản vá hoạt động ra sao?	17
5. Ít nhất ba cơ chế hardening	17
Câu hỏi bổ sung	17
Chương 16. Kiểm thử	17
Phạm vi kiểm thử	17
26_presentation_mode.png	17
27_pytest_passed.png	18
Chương 17. Kết luận	18
Kết quả và bài học	18
28_report_files.png	18
Phụ lục. Bằng chứng và câu lệnh	19
A. Mã nguồn và compiler flags	19
B. Request, response và trace	19
C. ASan và GDB	19
D. Câu lệnh chạy	19

Chương 1. Giới thiệu

Bối cảnh và mục tiêu

Buffer Overflow xuất hiện khi chương trình ghi vượt sức chứa của vùng nhớ đích. Trong kiến trúc web, input HTTP có thể đi qua gateway Python rồi kích hoạt lỗi ở module native C; vì vậy ranh giới HTTP không loại bỏ rủi ro memory corruption.

Lab quan sát luồng Browser -> HTTP POST -> Flask -> subprocess -> chương trình C, so sánh bản dùng strcpy với hai bản vá và binary hardened.

- Nhận biết nguyên nhân ghi vượt.
- Quan sát AddressSanitizer, signal/exit code và GDB local.
- Chứng minh defense in depth bằng kiểm tra ở browser, Flask, C và compiler.

Phạm vi và an toàn

Mọi thao tác chỉ chạy trên 127.0.0.1, Linux VM/WSL/Docker local. Input chỉ là văn bản bình thường hoặc ký tự lặp A/B; không có shellcode, ROP, reverse shell, persistence hay thay đổi return address.

Lab chỉ chứng minh crash/dừng có kiểm soát và bản vá. Ngưỡng crash phụ thuộc kiến trúc, ABI, compiler, flags và stack layout nên chỉ được kết luận từ bằng chứng của lần chạy thật.

01_home_overview.png

ẢNH THỦ CÔNG CẦN BỔ SUNG: 01_home_overview.png
URL hoặc lệnh: http://127.0.0.1:5002/
Thao tác: Mở trang chủ.
Nội dung bắt buộc: Mục tiêu, kiến trúc và các mode.

Hình 1. Tổng quan kiến trúc và chế độ thực hành.

Chương 2. Cơ sở lý thuyết

Bộ nhớ và chuỗi C

Buffer là vùng nhớ có kích thước hữu hạn. Stack lưu stack frame của lời gọi hàm, thường gồm biến local, metadata của frame và dữ liệu điều khiển; thứ tự cụ thể phụ thuộc ABI/compiler. Chuỗi C kết thúc bằng byte null, vì vậy name[32] chỉ chứa an toàn tối đa 31 byte dữ liệu.

Stack Buffer Overflow là một dạng memory corruption khi ghi vượt biến local trên stack. Vùng liền kề có thể bị thay đổi, dẫn đến kết quả sai, abort hoặc signal; không phải mọi lần ghi vượt đều crash ngay.

API xử lý chuỗi

strcpy, gets và sprintf không biết sức chứa đích theo cách đủ để ngăn ghi vượt. snprintf giới hạn số byte ghi nhưng vẫn phải kiểm tra giá trị trả về để phát hiện truncate. strncpy có thể không thêm null terminator khi nguồn quá dài, nên đổi tên hàm mà không kiểm tra không phải bản vá hoàn chỉnh.

Công cụ và hardening

AddressSanitizer chèn instrumentation để phát hiện truy cập bộ nhớ sai trong kiểm thử; GDB dừng tiến trình, xem frame/local/backtrace. Stack Canary phát hiện giá trị bảo vệ bị đổi trước khi return. ASLR ngẫu nhiên hóa vị trí vùng nhớ; PIE cho executable tham gia ASLR; DEP/NX đánh dấu vùng dữ liệu không thực thi; RELRO bảo vệ một số cấu trúc dynamic linking; FORTIFY_SOURCE bổ sung kiểm tra cho một số lời gọi thư viện.

Các cơ chế này giảm rủi ro hoặc tăng khả năng phát hiện, không thay thế việc loại bỏ strcpy không giới hạn.

Chương 3. Kiến trúc hệ thống

Ba tầng

Browser UI thu input và mode. Flask áp dụng request limit, allowlist mode và gọi subprocess.run bằng danh sách argument với shell=False, timeout và working directory cố định. Native processor nhận đúng một argument, in metadata rồi xử lý chuỗi. Hệ điều hành trả exit code hoặc signal; Flask chuẩn hóa kết quả thành trace và HTTP response.

- Browser: biểu mẫu, timeline và inspector.
- Flask: routing, validation, subprocess và trace.
- C/OS: buffer, copy, hardening, exit/signal.

Sequence diagram

Browser --POST /submit--> Flask Gateway

Flask --subprocess.run([binary, name], shell=False)--> C Processor

C Processor --exit code / signal / stderr--> Operating System

Operating System --kết quả tiến trình--> Flask Gateway

Flask Gateway --HTTP response + Trace ID--> Browser

Browser -> HTTP POST -> Flask -> subprocess.run -> C processor -> OS -> HTTP response

Sơ đồ 1. Sequence diagram khái niệm của hệ thống.

Chương 4. Thiết kế chương trình lỗi

Mã nguồn và nguyên nhân

Hàm `process_name` khai báo `char name[32]` rồi gọi `strcpy(name, user_input)`. `strcpy` tiếp tục copy cho tới byte null của nguồn nhưng không nhận kích thước `name`. Khi input có 32 byte dữ liệu, lần ghi null terminator đã vượt ít nhất một byte.

Sơ đồ stack trong UI/report chỉ là mô hình giáo dục: vùng argument, return address, saved frame pointer, canary (nếu có) và `name[32]`. Vị trí thực tế phải quan sát bằng GDB trong đúng binary.

```
int process_name(const char *user_input) {
    char name[32];
    strcpy(name, user_input);
    return 0;
}
```

Đoạn mã 1. Mã nguồn tối thiểu chứa thao tác `strcpy` có chủ đích.

Chương 5. Thực nghiệm input bình thường

Kịch bản Le Minh

Input `Le Minh` gồm 7 ký tự ASCII và 7 byte UTF-8. Kỳ vọng kỹ thuật là dữ liệu cùng null terminator nằm trong `name[32]`. Request, stdout, PID, exit code và thời gian chỉ được coi là kết quả thực nghiệm khi trace/log thật tồn tại.

- Kiểm tra POST /submit và mode.
- Đối chiếu stdout/exit code trong Native Inspector.
- Đối chiếu visualizer: `overflow_bytes = 0`.

Chưa có evidence/traces/normal_vulnerable.json; chưa kết luận kết quả chạy input bình thường.

02_normal_input_before_submit.png

ẢNH THỦ CÔNG CẦN BỔ SUNG: 02_normal_input_before_submit.png
URL hoặc lệnh: <code>http://127.0.0.1:5002/vulnerable</code>
Thao tác: Nhập Le Minh, chụp trước khi gửi.
Nội dung bắt buộc: Input, mode và giới hạn.

Hình 2. Input bình thường trước khi gửi.

03_normal_http_request.png

ẢNH THỦ CÔNG CẦN BỔ SUNG: 03_normal_http_request.png
URL hoặc lệnh: POST /submit
Thao tác: Gửi Le Minh, mở Request Inspector.
Nội dung bắt buộc: POST /submit, Content-Type, length và mode.

Hình 3. HTTP request của input bình thường.

04_normal_native_process.png

ẢNH THỦ CÔNG CẦN BỔ SUNG: 04_normal_native_process.png
URL hoặc lệnh: http://127.0.0.1:5002/vulnerable
Thao tác: Mở Native Process Inspector.
Nội dung bắt buộc: Exit code/stdout thực tế; dự kiến exit 0 cho Le Minh.

Hình 4. Kết quả tiến trình native với input bình thường.

05_normal_memory_visualizer.png

ẢNH THỦ CÔNG CẦN BỔ SUNG: 05_normal_memory_visualizer.png
URL hoặc lệnh: http://127.0.0.1:5002/vulnerable
Thao tác: Mở Memory Visualizer.
Nội dung bắt buộc: 7 byte dữ liệu, null và overflow 0.

Hình 5. Mô hình bộ nhớ không overflow.

Chương 6. Thực nghiệm input dài

Các mốc

31 byte dữ liệu vừa đủ cùng một null trong buffer 32 byte. 32 byte dữ liệu cần byte null thứ 33 nên đã ghi vượt về mặt mô hình chuỗi C, nhưng không được đồng nhất với crash. Các input 33, 64 và 128 byte dùng để quan sát ASan/exit/signal trong giới hạn lab.

Ba đại lượng phải tách biệt: độ dài đầu tiên ghi vượt theo capacity; độ dài đầu tiên ASan báo trong dữ liệu chạy thật; và độ dài đầu tiên tiến trình crash/dừng theo dữ liệu chạy thật.

Giới hạn kết luận

Báo cáo không khẳng định ngưỡng crash giống nhau trên mọi máy. Ký tự Unicode có thể chiếm nhiều byte UTF-8, nên validation và capacity phải dựa trên byte khi C copy byte.

Chưa có evidence/logs/length_test.json; ngưỡng ASan và crash chưa được quan sát.

Bảng 1. Phân biệt capacity, ASan và crash.

Mốc	Ý nghĩa	Cách xác nhận
31 byte	Capacity an toàn	Mô hình name[32] + null
32 byte	Ít nhất 1 byte ghi vượt	Mô hình copy tới null
ASan đầu tiên	Phát hiện instrumentation	length_test.json thật
Crash đầu tiên	Dừng tiến trình	exit/signal thật

06_overflow_32_input.png

ẢNH THỦ CÔNG CẦN BỔ SUNG: 06_overflow_32_input.png
URL hoặc lệnh: http://127.0.0.1:5002/vulnerable
Thao tác: Nhập A lặp 32.
Nội dung bắt buộc: 32 byte dữ liệu và giải thích null terminator.

Hình 6. Input 32 byte tại ranh giới chuỗi C.

07_overflow_32_memory_boundary.png

ẢNH THỦ CÔNG CẦN BỔ SUNG: 07_overflow_32_memory_boundary.png
URL hoặc lệnh: POST /submit
Thao tác: Gửi 32 byte, mở visualizer.
Nội dung bắt buộc: Ít nhất 1 byte ngoài biên; không khẳng định crash.

Hình 7. Ranh giới ghi vượt ở input 32 byte.

08_overflow_64_request.png

ẢNH THỦ CÔNG CẦN BỔ SUNG: 08_overflow_64_request.png
URL hoặc lệnh: POST /submit
Thao tác: Gửi A lặp 64 ở vulnerable_asan.
Nội dung bắt buộc: Request và input length 64 byte.

Hình 8. Request 64 byte tới profile ASan.

09_overflow_64_strcpy_step.png

ẢNH THỦ CÔNG CẦN BỔ SUNG: 09_overflow_64_strcpy_step.png
URL hoặc lệnh: http://127.0.0.1:5002/vulnerable
Thao tác: Chọn bước strcpy.
Nội dung bắt buộc: strcpy không nhận kích thước đích.

Hình 9. Bước copy không giới hạn của strcpy.

10_overflow_64_memory_visualizer.png

ẢNH THỦ CÔNG CẦN BỔ SUNG: 10_overflow_64_memory_visualizer.png
URL hoặc lệnh: http://127.0.0.1:5002/vulnerable
Thao tác: Mở visualizer của trace 64 byte.
Nội dung bắt buộc: Buffer 32 byte và vùng overflow.

Hình 10. Mô hình vùng ghi vượt với input 64 byte.

11_asan_detected.png

ẢNH THỦ CÔNG CẦN BỔ SUNG: 11_asan_detected.png
URL hoặc lệnh: http://127.0.0.1:5002/vulnerable
Thao tác: Mở ASan Inspector.
Nội dung bắt buộc: stack-buffer-overflow từ log thật.

Hình 11. AddressSanitizer phát hiện lỗi.

12_asan_stack_trace.png

ẢNH THỦ CÔNG CẦN BỔ SUNG: 12_asan_stack_trace.png
URL hoặc lệnh: http://127.0.0.1:5002/vulnerable
Thao tác: Mở chi tiết stack trace.
Nội dung bắt buộc: File, process_name và dòng strcpy.

Hình 12. Stack trace ASan định vị nguồn lỗi.

13_native_crash_result.png

ẢNH THỦ CÔNG CẦN BỔ SUNG: 13_native_crash_result.png
URL hoặc lệnh: POST /submit
Thao tác: Mở Native Inspector sau trace 64.
Nội dung bắt buộc: Exit code/signal thực tế.

Hình 13. Trạng thái dừng của tiến trình native.

14_final_vulnerable_verdict.png

ẢNH THỦ CÔNG CẦN BỔ SUNG: 14_final_vulnerable_verdict.png
URL hoặc lệnh: http://127.0.0.1:5002/vulnerable
Thao tác: Mở Final Security Verdict.
Nội dung bắt buộc: Overflow, detection/crash thực tế và nguyên nhân.

Hình 14. Kết luận bảo mật của bản vulnerable.

15_length_test_table.png

ẢNH THỦ CÔNG CẦN BỔ SUNG: 15_length_test_table.png
URL hoặc lệnh: python scripts/test_lengths.py
Thao tác: Chạy test độ dài và chụp bảng thật.
Nội dung bắt buộc: Các length, HTTP, exit/signal, ASan/crash.

Hình 15. Kết quả kiểm thử theo độ dài.

Chương 7. Phân tích GDB

Quy trình quan sát

Dùng binary vulnerable_debug với symbol -g, frame pointer và tối ưu O0. Break tại process_name, xem frame, info locals, sizeof(name), vùng byte quanh name, rồi tiếp tục để quan sát signal/backtrace. Địa chỉ chỉ xuất hiện trong terminal GDB local.

Phiên overflow chỉ dùng 64 ký tự A. Không sửa register, không ghi memory, không nhảy địa chỉ tùy ý và không có nội dung khai thác thực thi mã.

Giới hạn bằng chứng

Nếu evidence/gdb chưa có log, báo cáo ghi trạng thái chưa chạy; script .gdb và hướng dẫn không được xem là kết quả thực nghiệm.

Chưa có log GDB thật; chỉ có script/hướng dẫn để chạy local.

16_gdb_breakpoint.png

ẢNH THỦ CÔNG CẦN BỔ SUNG: 16_gdb_breakpoint.png
URL hoặc lệnh: <code>gdb -q -x gdb/inspect_normal.gdb</code>
Thao tác: Chụp khi dừng tại <code>process_name</code> .
Nội dung bắt buộc: Breakpoint, file và dòng.

Hình 16. GDB dừng tại `process_name`.

17_gdb_local_buffer.png

ẢNH THỦ CÔNG CẦN BỔ SUNG: 17_gdb_local_buffer.png
URL hoặc lệnh: <code>p sizeof(name); x/64bx &name</code>
Thao tác: Quan sát local buffer.
Nội dung bắt buộc: <code>sizeof(name)=32</code> hoặc vùng byte local.

Hình 17. Biến `local name[32]` trong GDB.

18_gdb_overflow_stop.png

ẢNH THỦ CÔNG CẦN BỔ SUNG: 18_gdb_overflow_stop.png
URL hoặc lệnh: <code>gdb -q -x gdb/inspect_overflow.gdb</code>
Thao tác: Chạy A lặp 64, chụp khi dừng.
Nội dung bắt buộc: Signal/trạng thái và backtrace thật.

Hình 18. Phiên overflow được quan sát bằng GDB.

Chương 8. Bản vá 1 - kiểm tra độ dài

Thiết kế

`secure_length` dùng `strlen` với giới hạn hợp lý. Nếu input vượt 31 byte, hàm trả exit code riêng trước mọi thao tác copy. Input hợp lệ được copy đúng số byte bằng `memcpy` và đặt `name[length] = '\0'`.

Kiểm tra lại ở C chứng minh defense in depth: bỏ qua thuộc tính `maxlength` của browser hoặc gọi thẳng endpoint vẫn không thể ép native copy quá capacity.

19_secure_length_reject.png

ẢNH THỦ CÔNG CẦN BỔ SUNG: 19_secure_length_reject.png
URL hoặc lệnh: <code>http://127.0.0.1:5002/secure/length</code>
Thao tác: Gửi A lặp 64.
Nội dung bắt buộc: Từ chối trước copy và exit code thực tế.

Hình 19. Bản vá length từ chối input dài.

20_secure_length_timeline.png

ẢNH THỦ CÔNG CẦN BỔ SUNG: 20_secure_length_timeline.png
URL hoặc lệnh: <code>http://127.0.0.1:5002/secure/length</code>
Thao tác: Chọn bước strlen/compare/reject.
Nội dung bắt buộc: Đo byte, so sánh 31 và không ghi buffer.

Hình 20. Timeline kiểm tra độ dài trước copy.

Chương 9. Bản vá 2 - snprintf

Thiết kế

`secure_snprintf` gọi `snprintf(name, sizeof(name), "%s", user_input)`. Nếu giá trị trả về âm, xử lý lỗi format/encoding; nếu lớn hơn hoặc bằng `sizeof(name)`, phát hiện truncate và từ chối thay vì chấp nhận tên bị cắt.

Giới hạn ghi ngăn overflow trong name, còn kiểm tra return value đảm bảo chính sách dữ liệu rõ ràng. Đây là lý do chỉ thay strcpy bằng snprintf mà bỏ qua return value vẫn chưa đủ.

21_secure_snprintf_reject.png

ẢNH THỦ CÔNG CẦN BỔ SUNG: 21_secure_snprintf_reject.png
URL hoặc lệnh: <code>http://127.0.0.1:5002/secure/snprintf</code>
Thao tác: Gửi A lặp 64, xem return value.
Nội dung bắt buộc: Phát hiện truncate và từ chối.

Hình 21. Bản vá snprintf từ chối truncate.

22_code_comparison.png

ẢNH THỦ CÔNG CẦN BỔ SUNG: 22_code_comparison.png
URL hoặc lệnh: http://127.0.0.1:5002/comparison
Thao tác: Mở ba cột so sánh.
Nội dung bắt buộc: strcpy, length check và snprintf return value.

Hình 22. So sánh mã vulnerable và hai bản vá.

Chương 10. Request limit

Defense in depth

Browser giới hạn thao tác thuận tiện, Flask đặt MAX_CONTENT_LENGTH và giới hạn name theo byte UTF-8, còn chương trình C tự áp capacity 31 byte cho hai bản vá. Mỗi lớp xử lý một ranh giới tin cậy khác nhau.

Chỉ giới hạn frontend không đủ vì client nằm ngoài vùng tin cậy và request có thể được tạo không qua form. Firewall cũng không hiểu điều kiện copy nội bộ; request hợp lệ về mạng vẫn có thể mang input dài tới backend native.

Chương 11. Compiler hardening

Cơ chế

Stack protector kiểm tra canary; PIE hỗ trợ ASLR cho executable; Full RELRO khóa relocation sau khởi tạo; NX giữ stack/data không thực thi; FORTIFY_SOURCE thêm kiểm tra khi compiler biết kích thước object. Các thuộc tính phải được xác minh bằng file/readelf/objdump trên binary thật.

vulnerable_debug chỉ tắt stack protector để quan sát và vẫn không dùng executable stack. ASan là profile kiểm thử. Các binary secure bật nhóm hardening nhưng secure coding vẫn là lớp chính.

Bảng 2. Profile build và nguyên tắc xác minh.

Profile	Mục đích	Trạng thái hardening
vulnerable_debug	GDB	Xác minh bằng file/readelf
vulnerable_asan	Phát hiện lỗi	Không đại diện production
secure_length	Bản vá length	Xác minh binary thật
secure_snprintf	Bản vá snprintf	Xác minh binary thật
secure_hardened	So sánh hardening	Xác minh binary thật

23_hardening_comparison.png

ẢNH THỦ CÔNG CẦN BỔ SUNG: 23_hardening_comparison.png
URL hoặc lệnh: http://127.0.0.1:5002/hardening
Thao tác: Mở Hardening Inspector sau make all.
Nội dung bắt buộc: Canary, PIE, RELRO, NX, FORTIFY từ công cụ thật.

Hình 23. So sánh thuộc tính hardening.

24_stack_canary_explanation.png

ẢNH THỦ CÔNG CẦN BỔ SUNG: 24_stack_canary_explanation.png
URL hoặc lệnh: http://127.0.0.1:5002/hardening
Thao tác: Mở giải thích canary.
Nội dung bắt buộc: Vị trí khái niệm và kiểm tra trước return.

Hình 24. Cơ chế Stack Canary.

25_asan_vs_hardening.png

ẢNH THỦ CÔNG CẦN BỔ SUNG: 25_asan_vs_hardening.png
URL hoặc lệnh: http://127.0.0.1:5002/hardening
Thao tác: Mở bảng ASan vs hardening.
Nội dung bắt buộc: Công cụ test khác lớp production.

Hình 25. Phân biệt ASan và hardening.

Chương 12. So sánh trước và sau vá

Kết quả thiết kế

Bản vulnerable copy không giới hạn nên input trên capacity tạo memory write ngoài name. Bản length từ chối trước copy; bản snprintf giới hạn write và từ chối truncate. Exit code của hai bản và thể hiện validation failure có kiểm soát, không phải crash.

Bảng 3. So sánh trước và sau vá.

Tiêu chí	Vulnerable	Secure length	Secure snprintf
Hàm copy	strcpy	memcpy sau validation	snprintf
Giới hạn	Không	Tối đa 31 byte	sizeof(name)
Input dài	Ghi vượt	Từ chối trước copy	Từ chối truncate

Tiêu chí	Vulnerable	Secure length	Secure snprintf
Hardening	Không thay bản vá	Lớp bổ sung	Lớp bổ sung

Chương 13. Mức độ ảnh hưởng

Tác động

Memory corruption có thể làm tiến trình crash, gây từ chối dịch vụ, làm sai dữ liệu hoặc rò rỉ dữ liệu trong các chương trình thực tế. Về lý thuyết, control data bị hỏng có thể đổi luồng điều khiển, nhưng lab không mô tả hay thực hiện quy trình khai thác.

Mức độ thực tế phụ thuộc dữ liệu kẻ buffer, quyền tiến trình, khả năng lặp lại, hardening và cách dịch vụ phục hồi. Tiến trình nên chạy least privilege và được giám sát/restart có kiểm soát.

Chương 14. Phòng chống

Biện pháp

Phòng chống bắt đầu bằng kiểm tra kích thước theo byte, API có giới hạn và kiểm tra return value. Request limit, compiler/OS hardening, sanitizer trong CI, fuzz testing, sandbox, least privilege, logging và monitoring tạo defense in depth.

Với parser nhạy cảm, ưu tiên thư viện đã được kiểm thử hoặc ngôn ngữ memory-safe khi phù hợp; hardening không phải lý do giữ lại thao tác copy không an toàn.

- Secure coding và validation tại ranh giới C.
- Sanitizer/fuzzing trong pipeline kiểm thử.
- Canary, PIE/ASLR, RELRO, NX, FORTIFY ở build/OS.
- Giảm quyền, sandbox, log và giám sát ở runtime.

Chương 15. Trả lời câu hỏi báo cáo

1. Buffer Overflow khác Injection như thế nào?

Buffer Overflow là vi phạm an toàn bộ nhớ do ghi vượt object; Injection xảy ra khi dữ liệu bị interpreter hiểu thành cú pháp/lệnh. Một lỗi làm hỏng memory layout, lỗi kia làm thay đổi ngữ nghĩa của ngôn ngữ đích như SQL hoặc shell.

2. Vì sao backend native bị kích hoạt qua HTTP?

HTTP chỉ là kênh vận chuyển. Flask trích trường name rồi truyền nó thành argument cho chương trình C; nếu C copy không kiểm tra, dữ liệu web đi qua các tầng và chạm trực tiếp buffer local.

3. Vì sao firewall không đủ?

Firewall kiểm soát kết nối/luồng mạng, không biết capacity của name[32] hay semantics của strcpy. Request tới đúng local service vẫn có thể chứa dữ liệu dài hợp lệ ở tầng HTTP.

4. Bản vá hoạt động ra sao?

Bản vá length đo byte và từ chối trên 31 trước copy. Bản vá snprintf giới hạn số byte ghi rồi kiểm tra return value để từ chối truncate. Cả hai giữ kiểm tra tại C thay vì chỉ tin browser/Flask.

5. Ít nhất ba cơ chế hardening

Stack Canary phát hiện giá trị bảo vệ bị đổi; PIE kết hợp ASLR làm địa chỉ executable thay đổi; DEP/NX ngăn thực thi tại vùng data. RELRO bảo vệ relocation và FORTIFY_SOURCE tăng kiểm tra thư viện khi compiler có đủ thông tin.

Câu hỏi bổ sung

Buffer name là biến local trong stack frame của process_name. Khi input vượt buffer, các byte ngoài object bị ghi và chương trình có undefined behavior. strcpy/gets/sprintf nguy hiểm vì không gắn thao tác với sức chứa đích; memory corruption nghiêm trọng vì có thể ảnh hưởng dữ liệu và control flow, không chỉ một điều kiện logic.

Chương 16. Kiểm thử

Phạm vi kiểm thử

Pytest bao phủ route/healthcheck, input bình thường, ASan overflow, hai bản vá, request limit và các ràng buộc an toàn. Native tests chạy binary thật; report/screenshot tests kiểm tra artifact và manifest mà không OCR.

Kết quả chỉ được trích từ evidence/logs/pytest.txt của lần chạy thật. Nếu file vắng hoặc không kết thúc thành công, báo cáo không tuyên bố toàn bộ test pass.

Dòng cuối log pytest thật: 14 passed in 1.45s

26_presentation_mode.png

ẢNH THỦ CÔNG CẦN BỔ SUNG: 26_presentation_mode.png
URL hoặc lệnh: http://127.0.0.1:5002/vulnerable
Thao tác: Bật Presentation Mode cho trace có sẵn.
Nội dung bắt buộc: Một bước chữ lớn và điều hướng.

Hình 26. Presentation Mode của timeline.

27_pytest_passed.png

ẢNH THỦ CÔNG CẦN BỔ SUNG: 27_pytest_passed.png
URL hoặc lệnh: <code>sh scripts/run_tests.sh</code>
Thao tác: Chụp tổng kết pytest thật.
Nội dung bắt buộc: Kết quả hiện tại; chỉ pass nếu exit 0.

Hình 27. Kết quả pytest trong WSL.

Chương 17. Kết luận

Kết quả và bài học

Thiết kế lab cho thấy một input HTTP có thể tới buffer C, cách null terminator tạo ghi vượt tại ranh giới 32 byte, và cách sanitizer/debugger hỗ trợ quan sát. Hai bản vá loại bỏ ghi vượt theo hai chính sách rõ ràng, còn hardening tạo lớp giảm thiểu bổ sung.

Giới hạn chính là signal/ngưỡng crash và stack layout phụ thuộc môi trường; ảnh GDB phải chụp thủ công. Hướng phát triển an toàn là mở rộng fuzz test có giới hạn, chạy sanitizer trong CI và tiếp tục giảm phạm vi code native.

28_report_files.png

ẢNH THỦ CÔNG CẦN BỔ SUNG: 28_report_files.png
URL hoặc lệnh: <code>python scripts/generate_report.py</code>
Thao tác: Liệt kê report sau khi tạo.
Nội dung bắt buộc: DOCX và PDF khác 0 byte.

Hình 28. Hai artifact báo cáo được sinh.

Phụ lục. Bằng chứng và câu lệnh

A. Mã nguồn và compiler flags

Mã nguồn nằm trong native/*.c và native/processor_common.h. Makefile là nguồn sự thật về flags; tên profile không thay cho kiểm tra binary.

B. Request, response và trace

Bằng chứng thật nằm trong evidence/requests, evidence/responses và evidence/traces. File vắng nghĩa là chưa thu thập.

C. ASan và GDB

ASan log nằm trong evidence/asan; GDB log thủ công nằm trong evidence/gdb. Không có log giả hoặc nội dung khai thác.

D. Câu lệnh chạy

- make all
- python app.py
- python scripts/test_lengths.py
- gdb -q -x gdb/inspect_normal.gdb
- gdb -q -x gdb/inspect_overflow.gdb
- pytest
- python scripts/check_screenshots.py
- python scripts/generate_report.py